# Pronalažanje optimalne putanje na 2D grid mapi zasnovano na GBFS algoritmu

Embedded operativni sistemi • Uroš Cvijić EE11/2022, Marko Horstmann EE27/2022 • septembar 2026.

## 1. Uvod

Projekat obuhvata potpunu vertikalu jednog hardverskog akceleratora: od SystemC modela i RTL opisa, preko verifikacije, do Linux drajvera i korisničke aplikacije koje ga koriste na stvarnoj razvojnoj ploči. Ovaj dokument pokriva poslednji, ugrađeno-sistemski deo te vertikale: kako se dizajn spušta na ploču, kako operativni sistem saznaje za njega i kako mu program iz korisničkog prostora pristupa.

Akcelerator rešava problem pretrage putanje na mapi veličine 100×100 polja. Svako polje može biti prohodno ili zid, a algoritam je pohlepna pretraga vođena heuristikom: iz trenutnog polja se procene sva susedna, pa se za sledeći korak bira polje sa najmanjom procenjenom udaljenošću do cilja. Svako obrađeno polje pamti odakle se u njega stiglo, pa se po dolasku do cilja putanja rekonstruiše unazad. Hardver obavlja iterativni deo algoritma, a rekonstrukciju putanje radi procesor.

Sa strane procesora akcelerator izgleda kao dva odvojena uređaja na AXI magistrali. Prvi je samo jezgro, sa četiri kontrolna registra dostupna preko AXI-Lite interfejsa i jednom prekidnom linijom. Drugi je blok memorije od 64 KB, vidljiv kroz standardni AXI BRAM Controller, u koji procesor upisuje mapu pre pretrage i iz kog čita rezultat posle nje. Ta podela, na kontrolni deo i memorijski deo, prenosi se kroz ceo softverski sloj: stablo uređaja ima dva čvora, drajver pravi dva uređaja u sistemu datoteka, a aplikacija ih otvara kao dva fajla.

| Komponenta | Verzija / opis |
|---|---|
| Ploča | Digilent Zybo (xc7z010clg400-1), 512 MB DDR3, kristal 50 MHz |
| Sinteza | Vivado 2024.1, board part digilentinc.com:zybo:part0:1.0 |
| Sistem | PetaLinux 2026.1, Linux kernel 6.18 (linux-xlnx) |
| Rootfs | Arch Linux ARM, moduli i izvor kernela iz PetaLinux slike |
| Softver | platform drajver (kernel modul), test aplikacija u C-u |

## 2. Spuštanje dizajna na ploču i PetaLinux

### 2.1. Od Vivada do opisa hardvera

Vivado projekat se gradi Tcl skriptom koja pakuje sopstvene IP blokove, sastavlja blok dizajn oko Zynq procesorskog sistema, dodeljuje adrese i pokreće sintezu. Rezultat koji zanima operativni sistem je XSA fajl: on sadrži bitstream, koji programira programabilnu logiku, i opis hardvera iz kog se kasnije generišu i FSBL i stablo uređaja.

Jedan detalj u ovom koraku koštao nas je nedelju dana rada, pa zaslužuje poseban paragraf. Obični Zybo i Zybo Z7 imaju isti čip, xc7z010, ali potpuno različitu konfiguraciju procesorskog sistema: obični Zybo ima kristal od 50 MHz i 512 MB memorije, dok Z7 ima 33,333 MHz i 1 GB, uz drugačiji raspored

MIO pinova. Vivado po izabranom čipu ne vidi razliku, pa ne javlja nikakvu grešku. Naš projekat je prvobitno nosio konfiguraciju za Z7. Posledica je bila potpuna tišina: FSBL pri pokretanju prvo podešava PLL i DDR kontroler prema tim vrednostima, i zaglavi se u toj inicijalizaciji pre nego što ijedan znak stigne do serijskog porta. Dioda DONE ostaje ugašena, konzola ne ispisuje ništa, i nijedna provera sa strane Linuxa ne pomaže, jer se do Linuxa nikad ne stigne. Rešenje je da se u projektu izabere pravi board part i da se konfiguracija procesorskog sistema preuzme iz njegovog preseta, umesto da se zadaje eksplicitnim vrednostima. Skripta sada proverava da je frekvencija kristala 50 MHz i prekida se pre sinteze ako nije.

## 2.2. PetaLinux tok rada

PetaLinux iz XSA fajla proizvodi sve četiri komponente sistema: FSBL i U-Boot upakovane sa bitstream-om u BOOT.BIN, kernel sa stablom uređaja u image.ub, poseban system.dtb, i root sistem datoteka. Konfiguracija prati vežbu 11: root sistem na EXT4 particiji SD kartice i uključen paket kernel-devsrc, koji u sliku dodaje izvor kernela neophodan za kompajliranje modula na samoj ploči. Root sistem datoteka na kartici je Arch Linux ARM.

## 2.3. Stablo uređaja i polje compatible

Uređaji na AXI magistrali nisu samoopisujući kao PCI ili USB. Procesor ne može da ih otkrije, pa mu neko mora reći da postoje, gde se nalaze i koji prekid koriste. To je uloga stabla uređaja: bootloader ga učita u memoriju, a kernel iz njega napravi listu platformskih uređaja i za svaki traži drajver koji ga ume opslužiti. Kriterijum uparivanja je polje compatible: string u čvoru stabla mora se slovo za slovo poklopiti sa stringom u tabeli drajvera.

PetaLinux stablo generiše automatski iz opisa hardvera. Za našu programabilnu logiku napravi čvor magistrale i u njemu po jedan čvor za svaki IP blok, sa adresom iz Address Editora u polju reg, brojem prekidne linije u polju interrupts i nizom xlnx parametara. Automatski dodeljeni compatible stringovi su izvedeni iz imena IP blokova: xlnx,gbfs-hardware-accelerator-1.0 za jezgro i xlnx,axi-bram-ctrl-4.1 za memorijski kontroler. Nijedan od njih drajver ne poznaje.

Generisani fajlovi se ne smeju menjati direktno, jer ih svaki sledeći build prepiše. Umesto toga, izmene se unose u system-user.dtsi, koji se uključuje poslednji i prepisuje samo polja koja u njemu navedemo. U našem slučaju to su tačno dva polja: compatible jezgra postavljeno na pathfinder i compatible memorijskog kontrolera na pathfinder_bram. Adrese i prekid ostaju kako ih je generator upisao, pa promena adresa u Vivadu ne zahteva nikakvu izmenu na strani softvera.

| Čvor u stablu | Adresa (reg) | Prekid | compatible |
|---|---|---|---|
| gbfs_hardware_accelerator@40010000 | 0x40010000, 4 KB | IRQ_F2P[0], level | pathfinder |
| axi_bram_ctrl@40000000 | 0x40000000, 64 KB | nema | pathfinder_bram |

Zamka koja se ovde lako previdi je da čvor u system-user.dtsi prepisuje postojeći samo ako se labela, ime i adresa u imenu poklope sa generisanim fajlom. Vežba 11 magistralu naziva amba_pl, a generator iz 2026.1 je naziva pl-bus. Ako se prepiše doslovno iz vežbe, dobija se novi, prazan čvor pod drugim imenom, originalni čvorovi zadrže xlnx stringove i drajver se nikad ne upari, bez ikakve poruke o grešci. Zato pre pisanja treba pogledati generisani pl.dtsi i prepisati tačna imena, a posle build-a dekompajlirati system.dtb i proveriti da compatible polja zaista glase kako treba.

Tip prekida u stablu mora odgovarati ponašanju hardvera. Generator upisuje level-triggered prekid, što je ispravno samo ako hardver drži liniju podignutom dok je softver ne obriše. Naš IP je prvobitno davao impuls od jednog takta, koji bi na takvom ulazu bio izgubljen; zato je jezgru dodat sticky fleg koji prekid drži aktivnim dok drajver ne upiše jedinicu u statusni registar, po istom principu kao AXI Timer iz vežbe 10.

### 2.4. SD kartica i pokretanje sistema

Kartica ima dve particije po uputstvu iz vežbe: malu FAT32 particiju BOOT i EXT4 particiju ROOT_FS, sa MBR tabelom particija, jer BootROM čipa Zynq-7000 ne čita GPT. Pri paljenju BootROM sa BOOT particije učita BOOT.BIN, FSBL iz njega inicijalizuje procesorski sistem, isprogramira FPGA bitstream-om i preda kontrolu U-Boot-u, koji preko boot.scr učita kernel i stablo iz image.ub i podigne Linux sa druge particije. Uspešno programiranje FPGA se vidi na diodi DONE, a ceo tok na serijskoj konzoli preko USB-UART porta ploče.

Postupak posle svake izmene hardvera je sada kratak i ponovljiv. Ako se menja samo logika, a adrese i prekid ostaju isti, dovoljno je novi bitstream upakovati u BOOT.BIN i prekopirati ga na karticu. Ako se menjaju adrese ili prekid, ponavlja se uvoz XSA arhive u PetaLinux projekat, build stabla i pakovanje, uz obaveznu proveru generisanog pl.dtsi zbog imena čvorova. Kernel i root sistem se pri tome ne diraju.

- BOOT particija: BOOT.BIN, boot.scr, image.ub, system.dtb
- ROOT_FS particija: Arch Linux ARM, moduli kernela, izvor kernela, drajver i aplikacija
- provera posle boot-a: čvorovi u /proc/device-tree/pl-bus i njihova compatible polja

## 3. Drajver i aplikacija

### 3.1. Struktura drajvera

Drajver je platform drajver, isti tip kao LED i timer drajveri sa vežbi. Kernelu se predstavlja tabelom compatible stringova i parom funkcija probe i remove. Kada pri učitavanju modula kernel u stablu nađe čvor sa odgovarajućim stringom, poziva probe sa strukturom koja opisuje taj uređaj. Probe iz nje izvuče fizičke adrese registara, rezerviše taj opseg da ga niko drugi ne bi zauzeo, preslika ga u virtuelni prostor kernela i, za jezgro, preuzme broj prekidne linije i registruje prekidnu rutinu. Remove sve to vraća obrnutim redosledom.

Jedan drajver opslužuje oba uređaja. U probe funkciji se po compatible stringu razlikuje da li je stigao čvor jezgra ili memorije, i podaci se smeštaju u jednu od dve globalne strukture. Isto razlikovanje koristi i remove. Time se izbegava pristup iz laboratorijskih primera, gde se uređaji razlikuju po redosledu pozivanja probe funkcije, koji zavisi od redosleda čvorova u stablu i lako se pokvari.

Ka korisničkom prostoru drajver izlaže dva znakovna uređaja sa zajedničkom tabelom operacija; svaka operacija prvo pogleda minor broj i prosledi posao funkciji za jezgro ili za memoriju. Uređaj /dev/pathfinder je kontrolni: upis tekstualne komande sa četiri koordinate upisuje početak i cilj u registre i pokreće pretragu, a čitanje blokira dok hardver ne javi prekid, sa vremenskim ograničenjem od deset sekundi.

| Ofset | Registar | Namena |
|---|---|---|
| 0x00 | START | koordinate početnog polja, x u bitima 13 do 7, y u bitima 6 do 0 |
| 0x04 | END | koordinate ciljnog polja, isti raspored |
| 0x08 | CTRL | bilo koji upis pokreće pretragu |
| 0x0C | STATUS | bit 0 završeno (briše se upisom jedinice), bit 1 zauzeto, bit 2 rešeno |

Uređaj /dev/pathfinder_bram je memorijski i ponaša se kao datoteka od 40000 bajtova: niz od 10000 tridesetdvobitnih reči, po jedna za svako polje mreže, redom po vrstama. Aplikacija ceo lavirint upiše jednim pozivom i ceo rezultat pročita jednim pozivom. Ovde drajver radi posao koji korisnik ne vidi: hardver mrežu drži u četiri banke po 2500 polja, a procesorska strana banke adresira sa korakom od 4096 reči, pa svaki indeks polja treba prevesti u pravu adresu. Ta funkcija prevođenja je jedino mesto u softveru koje zna kako je memorija fizički organizovana.

Prekidna rutina je kratka: pročita statusni registar, upiše ga nazad sa postavljenim bitom 0, čime obriše lepljivi fleg, i probudi proces koji čeka u čitanju. Uz to podržava i asinhrono obaveštavanje signalom SIGIO, po uzoru na treći zadatak sa vežbe 10, ako aplikacija ne želi da blokira.

## 3.2. Aplikacija

Aplikacija je program u C-u koji radi ono što bi u SystemC modelu radio procesor. Sastavi mrežu iz ugrađenog primera ili iz tekstualnog fajla sa sto redova po sto znakova, gde tačka označava prolaz, taraba zid, a S i E početak i cilj. Svako polje kodira u format koji hardver očekuje: devet bita za procenjenu udaljenost, po sedam bita za koordinate roditelja i jedan bit posećenosti, sa rezervisanim vrednostima za zid, početak i cilj.

Tok rada je linearan: upis mreže u memorijski uređaj, upis komande u kontrolni uređaj, blokirajuće čitanje koje se vraća kada stigne prekid, čitanje mreže nazad, i rekonstrukcija putanje praćenjem roditeljskih pokazivača od cilja ka početku. Pošto hardver roditelja upisuje isključivo u susedno polje, prekid lanca susednosti pouzdano označava da cilj nije dostignut, bez potrebe za posebnim statusnim bitom. Na kraju se mreža ispiše, sa zvezdicom za putanju i zarezom za posećena polja.

Aplikacija ima i režim bez hardvera, u kom istu mrežu obradi softverski model automata iz RTL opisa. To je omogućilo da se logika rekonstrukcije proveri na računaru pre nego što je ploča proradila, i da se posle uporedi izlaz hardvera sa modelom.

## 3.3. Rezultati

Po učitavanju modula kernel uparuje oba uređaja, rezerviše adrese 0x40000000 i 0x40010000 i registruje prekidnu liniju 61 kao level-triggered. Na ugrađenom lavirintu, sa zidovima u kolonama 20, 40, 60 i 80 i naizmeničnim prolazima, hardver nalazi putanju od 531 koraka za oko deset milisekundi od upisa komande do prekida. Softverski model daje istih 531 koraka, čime je potvrđeno da hardverska implementacija daje rezultat identičan referentnom.

Tokom integracije su ispravljene i dve greške u samom hardveru koje bi bez softverskog sloja ostale neprimećene. AXI BRAM Controller izbacuje bajt adresu, a adapterski blok ju je tumačio kao adresu reči, pa je procesor pogađao svaku četvrtu lokaciju i pogrešne banke. Prekid od jednog takta zamenjen je lepljivim flegom, jer se sa Linux strane impuls od deset nanosekundi ne može ni prozivati ni

pouzdano uhvatiti. Obe ispravke su male po obimu, ali pokazuju vrednost završetka vertikale: tek pravi drajver na pravoj ploči otkriva pretpostavke koje simulacija i verifikacija nisu dovele u pitanje.

# 4. Zaključak

Cilj EOS dela projekta bio je da akcelerator postane uređaj koji operativni sistem poznaje i koji običan program može da koristi kroz standardne operacije nad datotekama. To je postignuto: stablo uređaja opisuje oba dela akceleratora, drajver ih uparuje po compatible poljima i izlaže kroz dva znakovna uređaja, a aplikacija kroz njih rešava lavirint i dobija isti rezultat kao referentni model.

Najveća pouka tiče se granice između hardvera i softvera. Sva tri problema koja su nas najviše koštala, pogrešan board preset, bajt adresiranje u BRAM adapter bloku i prekid od jednog takta, bila su nevidljiva u simulaciji i verifikaciji, jer tamo nema procesora, bootloadera ni operativnog sistema koji bi ih doveli u pitanje. Tek pokušaj da se dizajn stvarno pokrene otkrio je pretpostavke ugrađene u projekat.